

## LECTURE 1

# Intro to CSCI 3022

Intro to the data science lifecycle and exploratory data analysis with Pandas

**CSCI 3022 @ CU Boulder**

Dr. Osita Onyejekwe

Content credit: [Acknowledgments](#)



## Lesson Learning Objectives:

- **Meet your classmates**
- **Name and explain the stages of the data science lifecycle**
- **Identify 5 key data properties to consider when doing Exploratory Data Analysis and implement using demo data**
- **Read in data from a .csv file to a Pandas DataFrame**

# Roadmap

---

Lecture 01, CSCI 3022

- [Intros & Logistics](#)
- [What is Data Science?](#)
- [Exploratory Data Analysis & Wrangling](#)
- Intro to Pandas
  - [Jupyter Notebook Demo: EDA using Pandas](#)
- [Supporting Materials:](#)
  - Intro to Pandas Data Structures

- NAME, YEAR, MAJOR, HOMETOWN
- HOBBIES/INTERESTS
- SOME RANDOM FUN FACT ABOUT YOU

## Canvas - Modules - Lesson Materials

2025 Spring Term

- [Home](#)
- [Announcements](#)
- [Modules](#)**
- [Assignments](#)
- [Gradescope](#)
- [Piazza](#)
- [Grades](#)
- [NameCoach](#)
- [Web Grading Sync](#)
- [Quizzes](#) 
- [Syllabus](#) 
- [Discussions](#) 
- [Pages](#) 
- [Outcomes](#) 

⋮ ▾ Lesson Materials - Week 1

⋮ Lesson 1 - Intro to Data Science Lifecycle

⋮  Lesson 1 - In Class Slides: Intro to Data Science Lifecycle

⋮  [Lesson 1 - In Class Jupyter Demo](#) 

⋮ Lesson 1 Supporting Materials

⋮  [Reading - The Data Science Lifecycle \(LDS 1.1-1.3\)](#) 

⋮  [Reading: Pandas DataFrames \(LDS: Chapter 6 Intro\)](#) 

⋮  [Reading: Data Scope & Question \(LDS 6.1.1 & 6.1.2\)](#) 



## Canvas -> Assignments

≡ CSCI 3022 > Assignments > HW 1

2025 Spring Term

[Home](#)

[Announcements](#)

[Modules](#)

[Assignments](#)

[Gradescope](#)

[Piazza](#)

[Grades](#)

[NameCoach](#)

[Web Grading Sync](#)

[Quizzes](#) 

[Syllabus](#) 

[Discussions](#) 

[Pages](#) 

[Outcomes](#) 

## HW 1

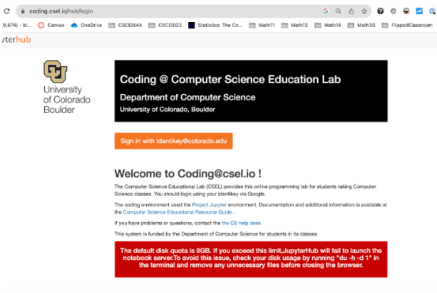
**Homework 1 is due: 11:59 PM Thursday Jan 16th.**

All HW is completed in Jupyter notebooks (.ipynb file).

[Use this link to access your assignment](#)  (this will open the assignment in a new tab)

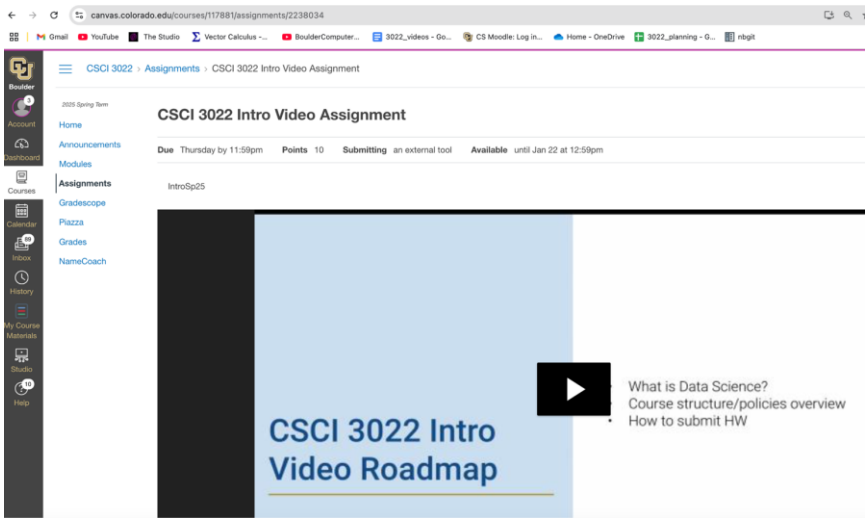
Here screenshots of what you should see after clicking the link above:

- 1). The link above will first redirect you here. Sign in using your id directly to coding.csel)



2). Select "CSCI 3022"

## HW 1 Includes an Intro Video Assignment



## Course Prerequisites (minimum grade C-)

---

- Data Structures (CSCI 2270 or equivalent)
- Calculus 2 (APPM 1360 or MATH 2300 or equivalent)
- Discrete Math (CSCI 2824 or equivalent)

# What is Data Science?

---

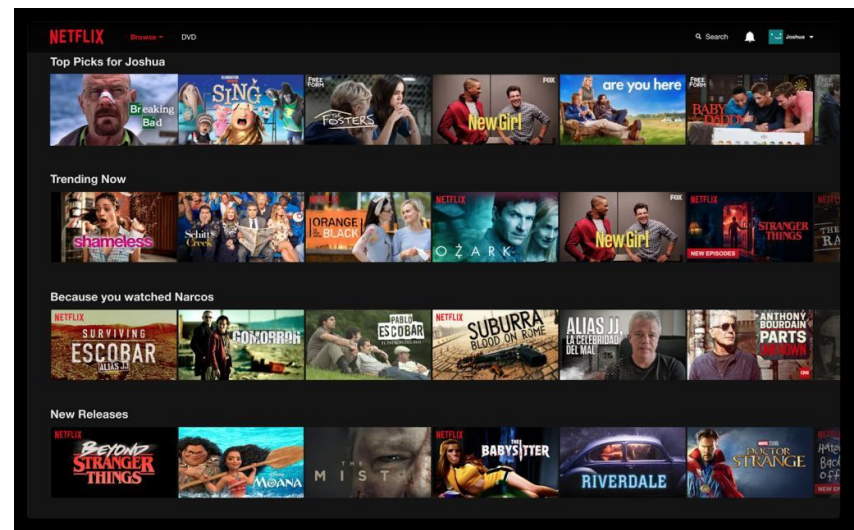
Lecture 01, CSCI 3022

- Intros & Logistics
- **What is Data Science?**
- Exploratory Data Analysis & Wrangling
- Pandas Data Structures
- Jupyter Notebook Demo: EDA using Pandas

# Some examples of Data Science?



# Recommendation Systems



| Top picks for you                                                                                                                                                                                                        |                                                                                                                                                                                                                                 |                                                                                                                                                                                                                |                                                                                                                                                                                                                       |                                                                                                                                                                                                                         |                                                                                                                                                                                                                         |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <p>Aviation Metal &amp; Alloys Pure Titanium Wire 0.50mm x 5M For Medical Uses or High Strength...</p> <p>★★★★☆ 13</p> <p>₹701.00</p> |  <p>Sabine's Notebook: In Which the Extraordinary Correspondence of Griffin &amp; Sabine Continues (Griffin and Sabine)</p> <p>★★★★★ 167</p> |  <p>Invento 1pcs Al Aluminium Alloy 2mm Plate/Sheet...</p> <p>★★★★☆ 37</p> <p>₹290.00</p> <p>Prime FREE Delivery</p>        |  <p>IBELL Angle Grinder AG10-70, 850W, Copper Armature, Disc...</p> <p>★★★★★ 1,744</p> <p>₹1,706.00</p> <p>prime FREE Delivery</p> |  <p>IBELL 200-89 Inverter ARC Compact Welding Machine...</p> <p>★★★★★ 1,723</p> <p>₹5,393.00</p> <p>prime FREE Delivery</p>          |  <p>GVD PVC &amp; FR Insulated 2 Core 1mm Length-10Mts: Flexible Copper Wires &amp; Cables for...</p> <p>★★★★☆ 12</p> <p>₹572.00</p> |
|  <p>TheGiftkart Transparent Crystal Clear Back Cover for Samsung...</p> <p>★★★★☆ 6,571</p> <p>₹199.00</p> <p>prime FREE One-Day</p>    |  <p>I Am a Strange Loop</p> <p>★★★★★ 589</p>                                                                                                 |  <p>HUPSHY Samsung Galaxy M21 2021 Armour Back Cover [...]</p> <p>★★★★☆ 1,738</p> <p>₹185.00</p> <p>prime FREE Delivery</p> |  <p>THE IDEA FACTORY: Bell Labs and the Great Age of American Innovation</p> <p>★★★★★ 565</p>                                      |  <p>Stookin N20 3.7V - 6V 100 RPM Micro Gear Reduction DC Motor with 30:1 Metal Gearbox For RC...</p> <p>★★★★☆ 76</p> <p>₹349.00</p> |  <p>Metamagical Themes: Questioning For The Essence Of Mind And Pattern</p> <p>★★★★★ 69</p>                                          |

# DJ Patil calls data scientists ‘a new kind of first responder’

By Rachel Leven | April 13, 2023

<https://www.youtube.com/watch?v=LiHMrn2AHpw>



DJ Patil spoke to UC Berkeley data science students on April 10. (Photo/

On March 14, 2020, the United States was on the brink of a pandemic. Covid-19 had killed at least 60 people and two cruise ships with ill passengers were set to dock in San Francisco. That's when DJ Patil received a call: How can data help California combat this?

So the former White House chief data scientist put together a plan. His team acquired hospital and community data, developed surveys, models, dashboards and data catalogs, and used those tools and insights to inform public officials across the state and the country.

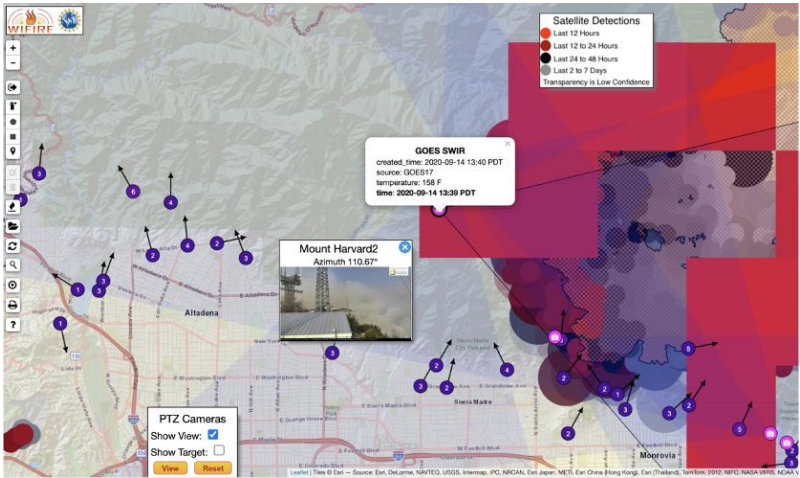
"All of this came together in this effort to really take on Covid," said Patil, who is now a general partner at GreatPoint Ventures, at an April 10 UC Berkeley



## First U.S. Chief Data Scientist (Obama Adm.)

<https://www.sciencefriday.com/segments/an-exit-interview-with-u-s-chief-data-scientist-dj-patil/>

# WIFIRE (UCSD) - Wildfire modeling and management



<https://wifire.ucsd.edu>



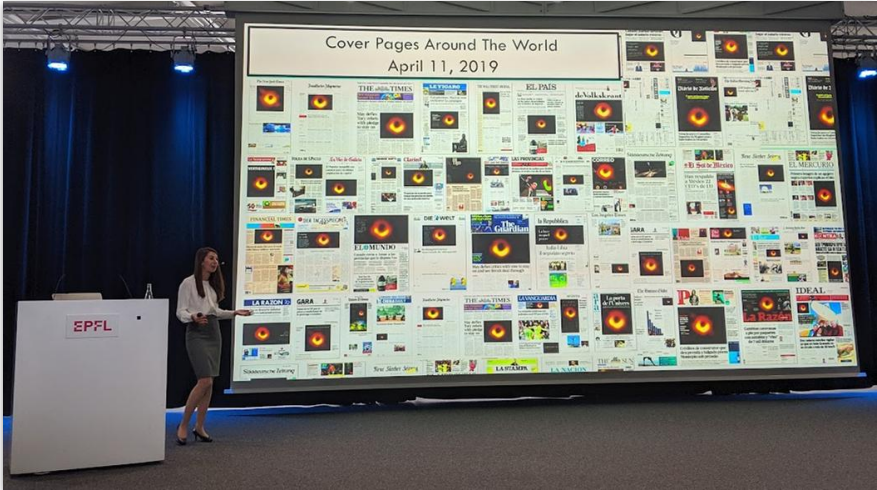
**WIFIRE**  
Cyberinfrastructure  
Integration



- Use-Inspired Translational Products
- Scalable Use Programs
- Novel AI Innovations



# 2019: First Image of a Black Hole



Katie Bouman  
MIT/Caltech



Talk Video: <https://youtu.be/TSgpliktqwc>

## THE ASTROPHYSICAL JOURNAL LETTERS

### First M87 Event Horizon Telescope Results. III. Data Processing and Calibration

The Event Horizon Telescope Collaboration, Kazunori Akiyama<sup>1,2,3,4</sup>, Anton Alberdi<sup>5</sup>, Walter Alef<sup>6</sup>, Keiichi Asada<sup>7</sup>, Rebecca Azulay<sup>8,9,6</sup>, Anne-Kathrin Baccko<sup>6</sup>, David Ball<sup>10</sup>, Mislav Baloković<sup>4,11</sup>, John Barrett<sup>2</sup> + Show full author list

Published 2019 April 10 • © 2019. The American Astronomical Society.  
[The Astrophysical Journal Letters, Volume 875, Number 1](#)

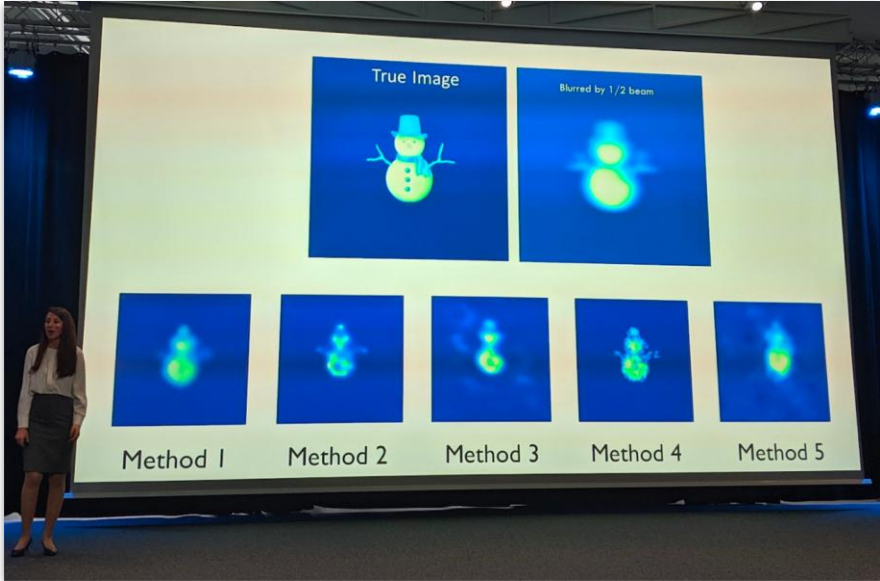
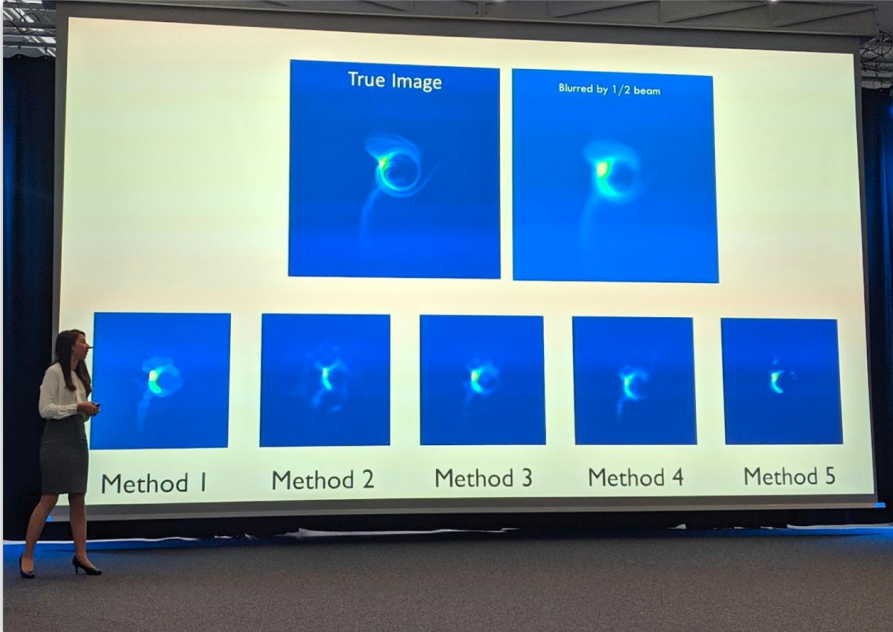
Software: DiFX (Deller et al. 2011), CALC, PolConvert (Martí-Vidal et al. 2016), HOPS (Whitney et al. 2004), CASA (McMullin et al. 2007), AIPS (Greisen 2003), ParselTongue (Kettenis et al. 2006), GNU Parallel (Tange 2011), GILDAS, cht-imaging (Chael et al. 2016, 2018), Numpy (van der Walt et al. 2011), Scipy (Jones et al. 2001), Pandas (McKinney 2010), Astropy (The Astropy Collaboration et al. 2013, 2018), Jupyter (Kluyver et al. 2016), Matplotlib (Hunter 2007).

22,953

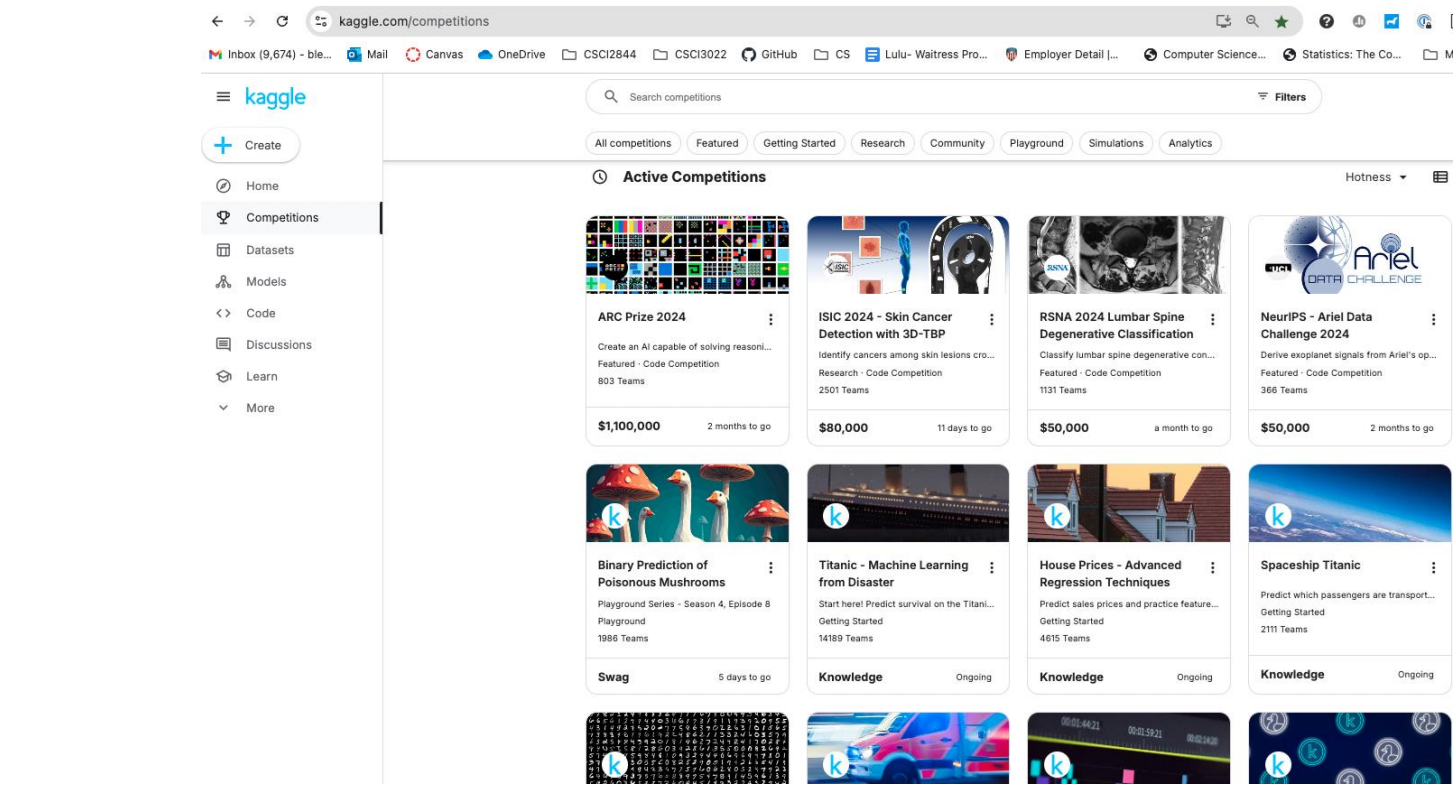
### Community Contributors

Analysis Courtesy of GitHub

# 2019: First Image of a Black Hole



# Kaggle: More Data Science Sets and Competitions



<https://www.kaggle.com/competitions>

# List of Topics to be Covered in CSCI 3022

- **Exploring, Cleaning and Visualizing Data**

- Intro to Pandas and NumPy
- Exploratory Data Analysis
- Wrangling Data
- Visualizing Data using matplotlib, seaborn & plotly

- **Probability & Statistics for Data Science**

- Independence
- Conditioning and Bayes Theorem
- Discrete and Continuous Random Variables
- Distributions and Joint Distributions
- Expectation & Variance
- Central Limit Theorem
- Sampling
- Using Statistical Simulation To Draw Inferences from Data:
  - Hypothesis and A/B Testing
  - Bootstrapping Confidence Intervals

- **Modeling/Intro to Machine Learning**

- Model design and loss formulation
- Simple Linear Regression
- Multiple Linear Regression
- Logistic Regression
- Feature Engineering
- Cross-Validation
- Regularization

matplotlib

SciPy

plotly

pandas

jupyter

scikit-learn

NumPy

Seaborn

## Lesson Learning Objectives:

- Name and explain the stages of the data science lifecycle

# The Data Science Lifecycle

---

Lecture 01, CSCI 3022

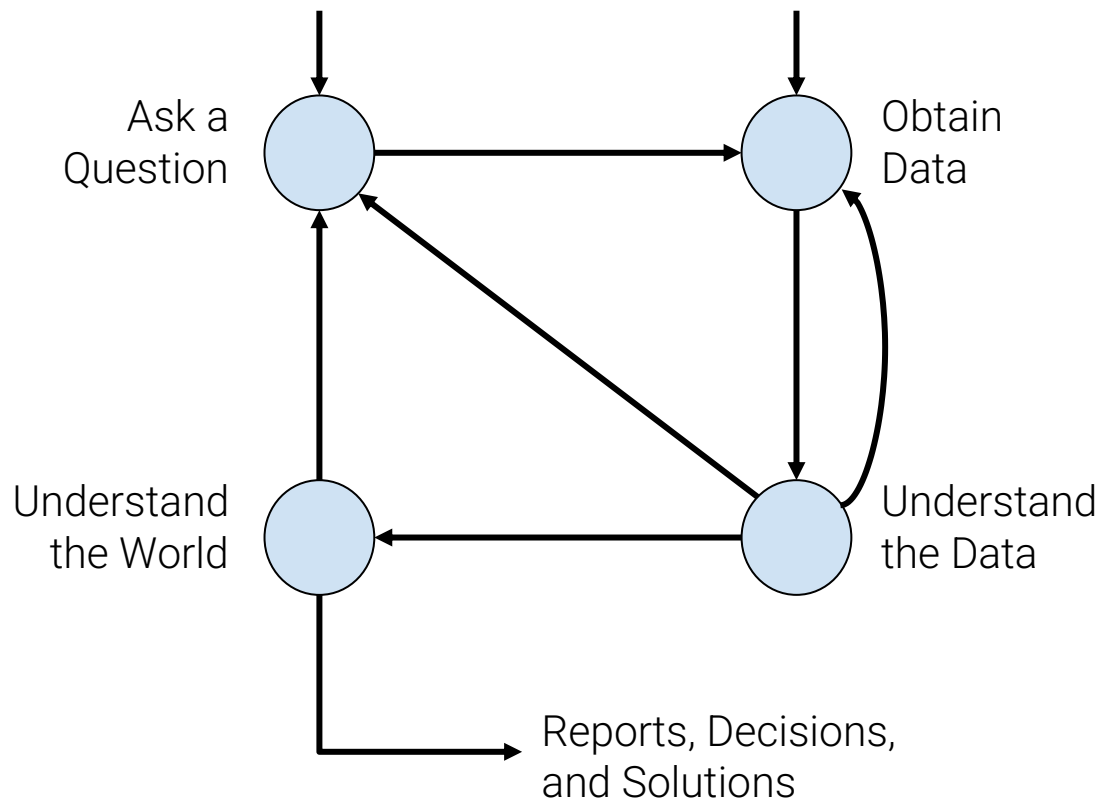
- Intros & Logistics
- What is Data Science?
  - **Data Science Lifecycle**
- Exploratory Data Analysis & Wrangling
- Intro to Pandas
  - Jupyter Notebook Demo: EDA using Pandas



# Data science lifecycle

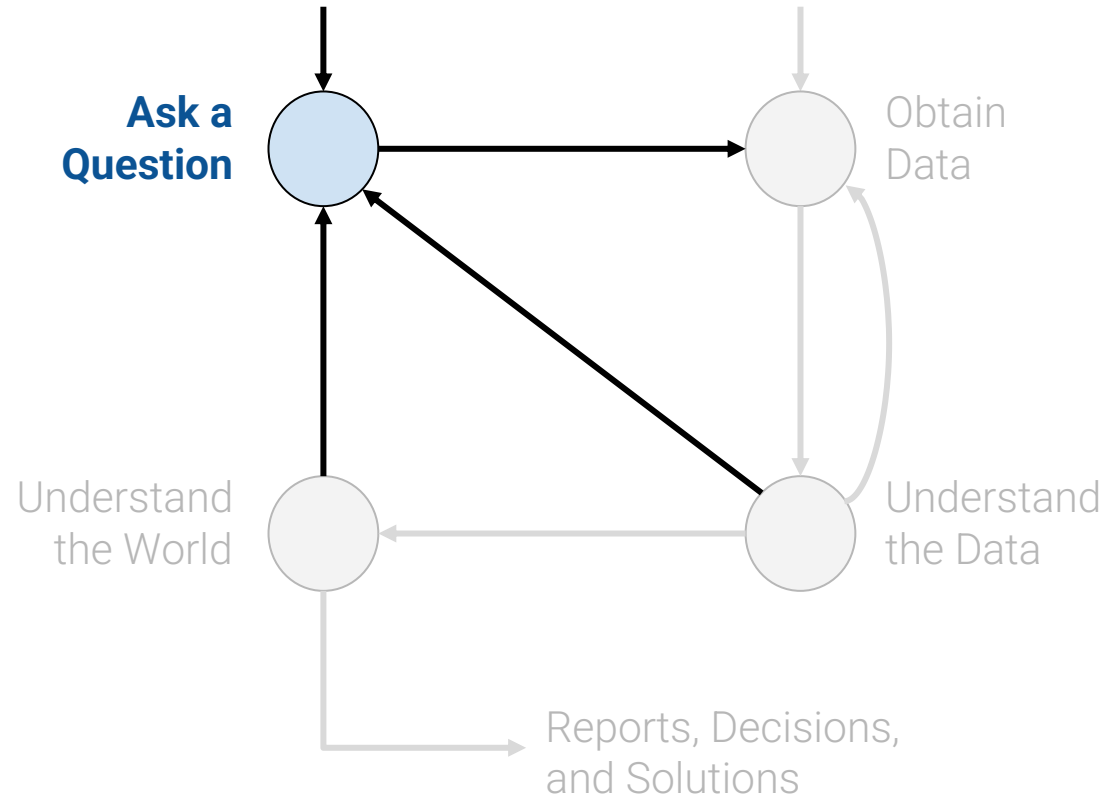
The data science lifecycle is a **high-level description** of the data science workflow.

Note the two distinct entry points!



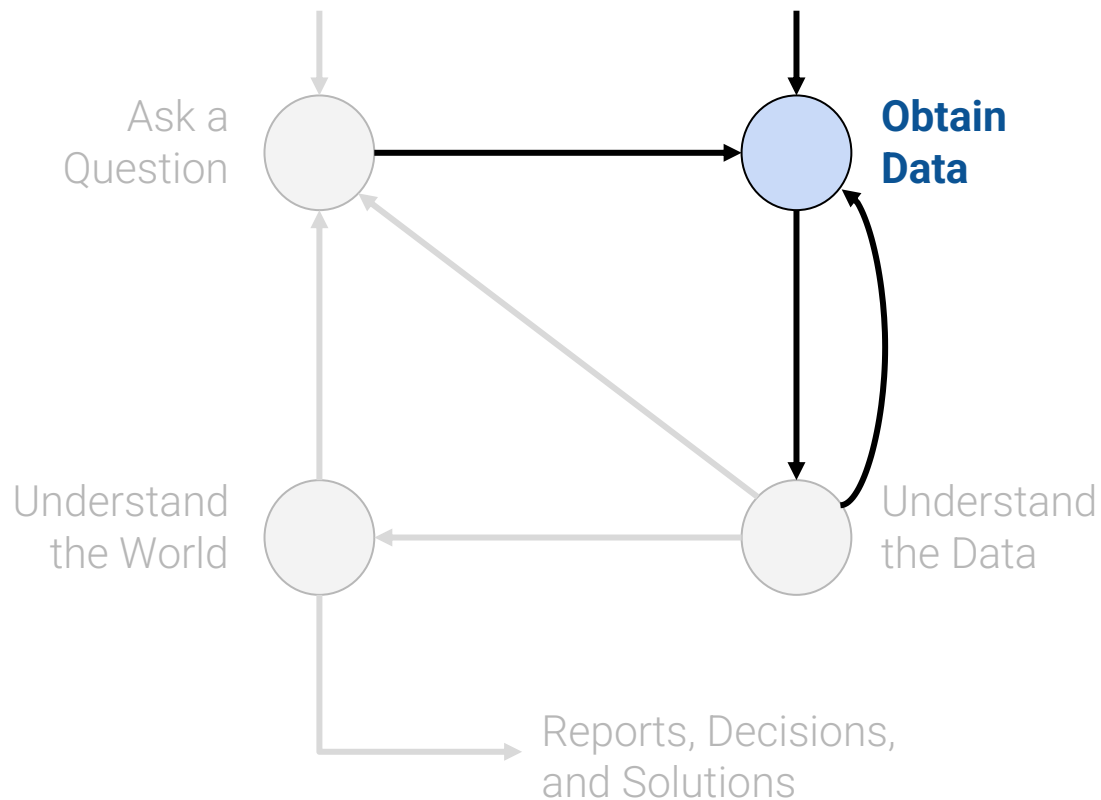
# 1. Question/Problem Formulation

- What do we want to know?
- What problems are we trying to solve?
- What are the hypotheses we want to test?
- What are our metrics for success?



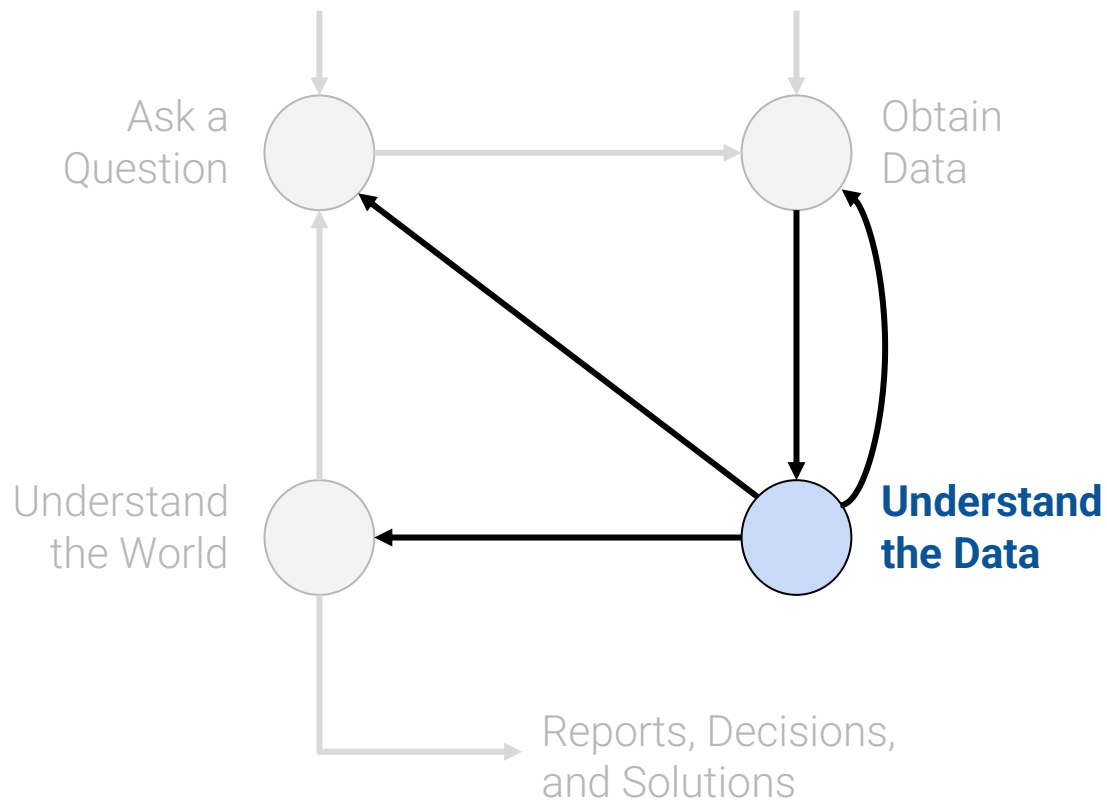
## 2. Data Acquisition and Cleaning

- What data do we have and what data do we need?
- How will we sample more data?
- Is our data representative of the population we want to study?



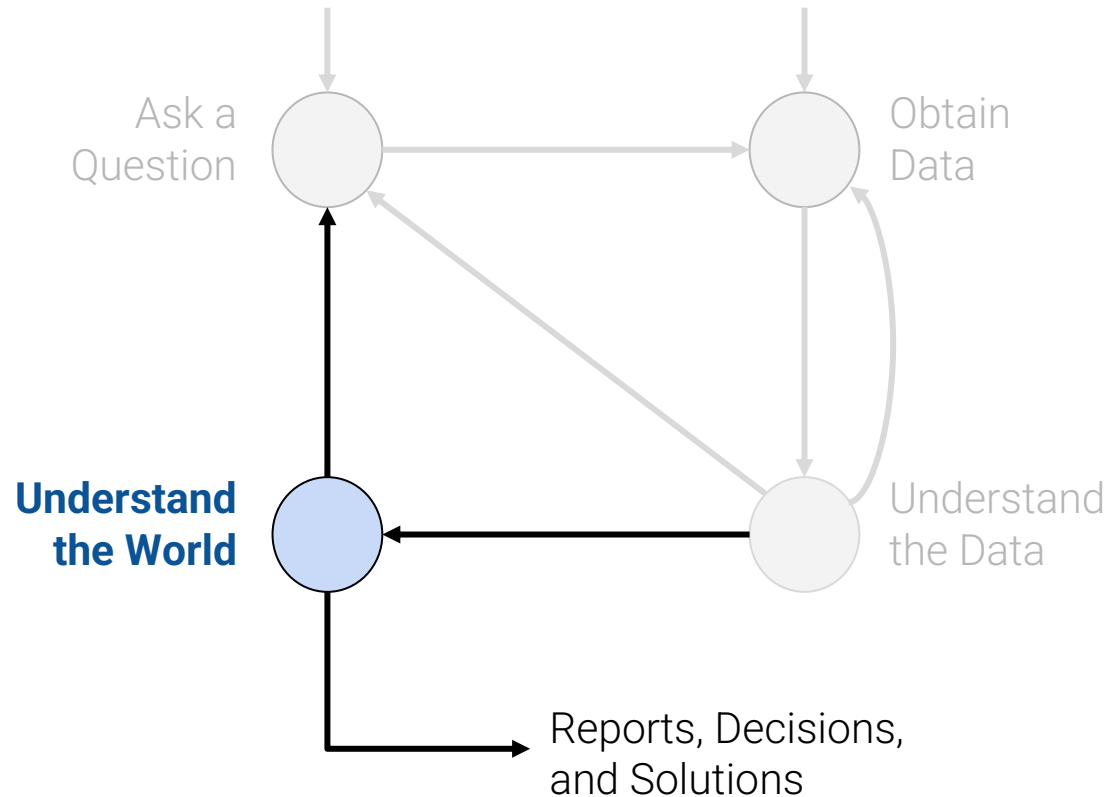
### 3. Exploratory Data Analysis & Visualization

- How is our data organized and what does it contain?
- Do we already have relevant data?
- What are the biases, anomalies, or other issues with the data?
- How do we transform the data to enable effective analysis?

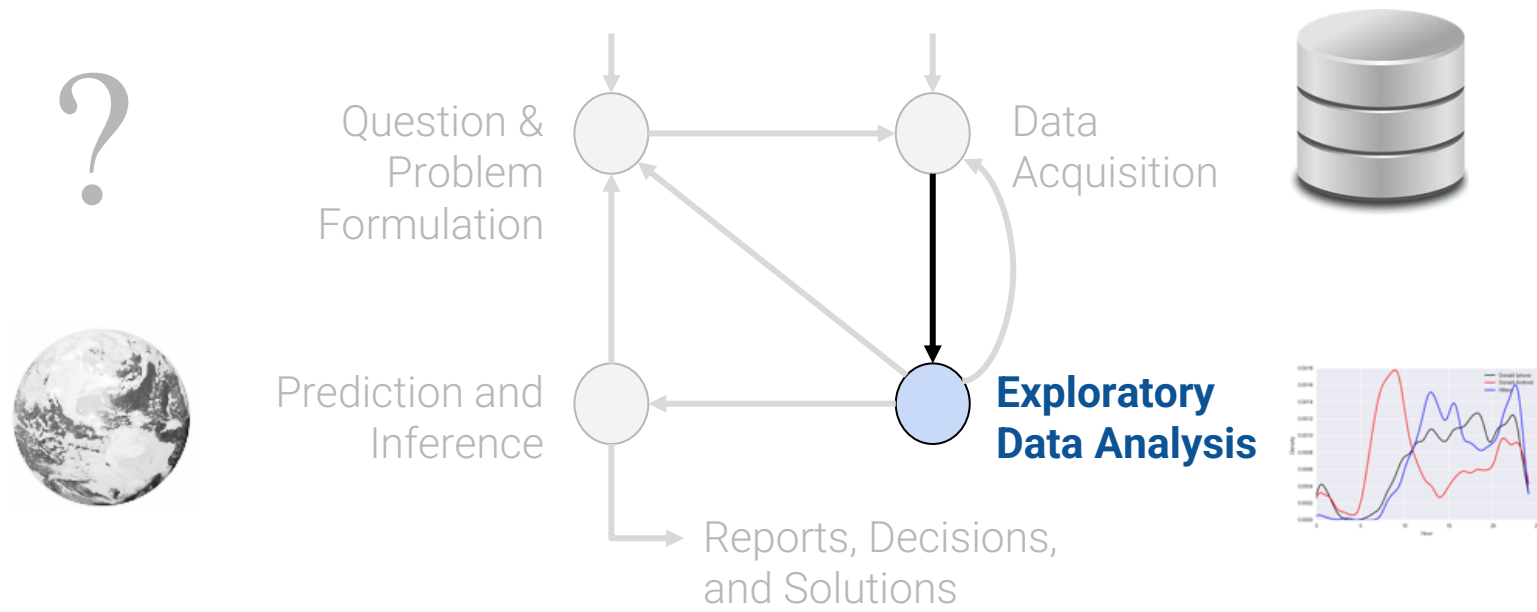


## 4. Prediction and Inference

- What does the data say about the world?
- Does it answer our questions or accurately solve the problem?
- How robust are our conclusions and can we trust the predictions?



# Plan for first few weeks



**(Weeks 1)**  
EDA, Wrangling and Data Visualization

## Lesson Learning Objectives:

- Meet your classmates
- Name and explain the stages of the data science lifecycle
- Identify 5 key data properties to consider when doing Exploratory Data Analysis and implement using demo data

# EDA & Wrangling

---

Lecture 01, CSCI 3022

- Intros & Logistics
- What is Data Science?
- **Exploratory Data Analysis & Wrangling**
- Intro to Pandas:
  - Jupyter Notebook Demo: EDA using Pandas





## Congratulations!!!

You **have collected** or **have been given** a box of data.

What do you do next?

## One Option: Exploratory Data Analysis (EDA)

*"Getting to know and understand the data"*

The process of **transforming**, **visualizing**, and **summarizing** data to:

- Build/confirm understanding of the data and its **provenance**
- Identify and address potential issues in the data.
- Inform the subsequent analysis.
- Discover *potential* hypotheses ...

*Provenance*: origin of data;  
methodology by which data  
were produced

**EDA is an open-ended analysis.**

- Informal, no specific idea of what we are looking for.
- Be willing to find something surprising!

Contrast with **confirmatory analysis**:

- Questions are fixed in advance.
- Allows for more rigorous statistical analysis.

## Another Option: Data Wrangling (Data Cleaning)

### Data Wrangling, or Data Cleaning:

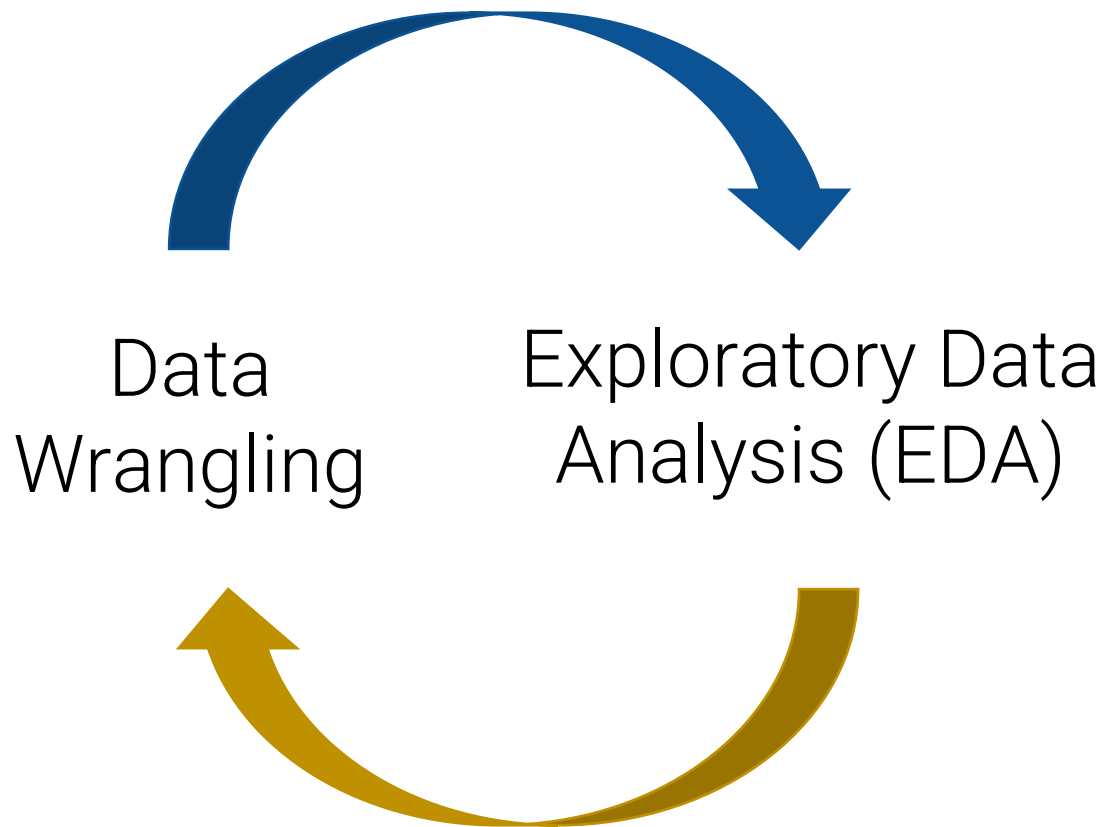
The process of transforming **raw data** to facilitate subsequent analysis.

Often addresses **issues** like...

- structure / formatting
- missing or corrupted values
- unit conversion
- encoding text as numbers
- ...

Sadly, data cleaning is a big part of data science...







**EDA is unboxing  
for data!**

## Exploratory Data Analysis (EDA) Guiding Principles

*“Exploratory data analysis is an attitude, a state of flexibility, a willingness to look for those things that we believe are not there, as well as those that we believe to be there.”* — John Tukey

# Key Data Properties to Consider in EDA

**Structure** -- the “shape” of a data file

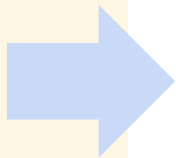
**Granularity** -- what does each record represent?

**Scope** -- how (in)complete is the data

**Temporality** -- how is the data situated in time

**Faithfulness** -- how well does the data capture “reality”

File Format  
Variable Type  
Multiple files  
(Primary and Foreign Keys)



**Structure** -- the “shape” of a data file

**Granularity** -- how fine/coarse is each datum

**Scope** -- how (in)complete is the data

**Temporality** -- how is the data situated in time

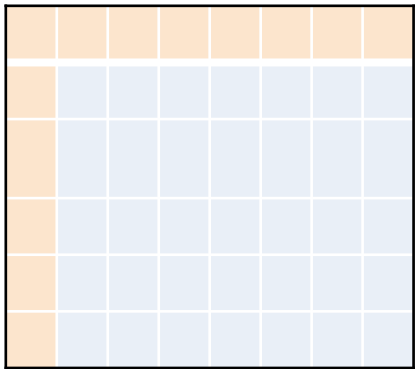
**Faithfulness** -- how well does the data capture “reality”

# Rectangular and Non-rectangular Data

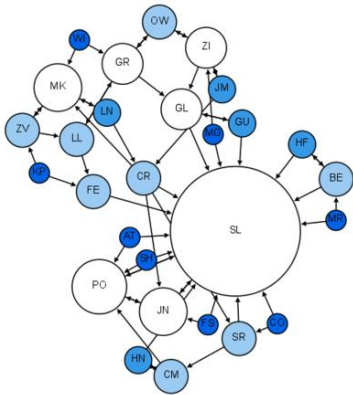
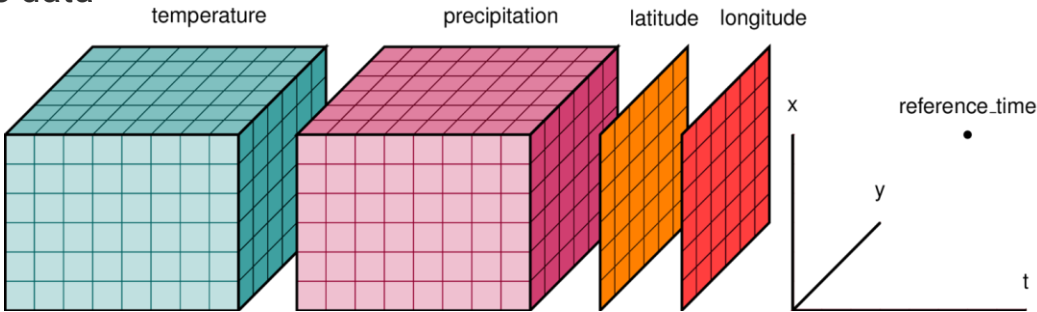
Data come in many different shapes.

Dataset's *structure*: Mental representation of the data

Tabular (rectangular) data



Non-rectangular data



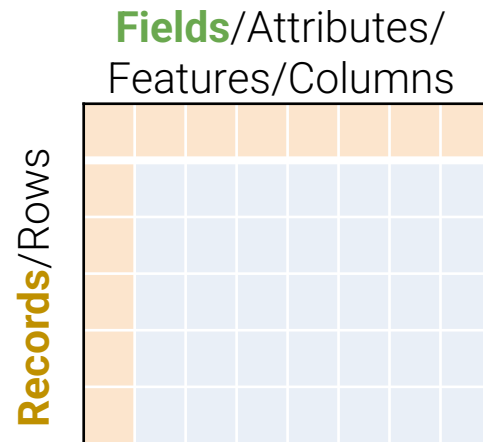


# Tabular/Rectangular Data

We often prefer tabular data for data analysis (why?)

- Regular structures are easy to manipulate and analyze
- A big part of data cleaning is about transforming data to be more rectangular

Two kinds of tabular data: **Tables** and **Matrices**.



**Tables** (a.k.a. **DataFrames** in R/Python and relations in SQL)

- Named columns with different types
- Manipulated using Pandas data transformation languages (map, filter, group by, join, ...)

**Matrices**

- Numeric data of the same type (float, int, etc.)
- Manipulated using linear algebra

# Data Scientists Love Rectangular/Tabular Data

"Tabular= data in a table.

|     | Year | Candidate         | Party                 | Popular vote | Result | %         |
|-----|------|-------------------|-----------------------|--------------|--------|-----------|
| 0   | 1824 | Andrew Jackson    | Democratic-Republican | 151271       | loss   | 57.210122 |
| 1   | 1824 | John Quincy Adams | Democratic-Republican | 113142       | win    | 42.789878 |
| 2   | 1828 | Andrew Jackson    | Democratic            | 642806       | win    | 56.203927 |
| 3   | 1828 | John Quincy Adams | National Republican   | 500897       | loss   | 43.796073 |
| 4   | 1832 | Andrew Jackson    | Democratic            | 702735       | win    | 54.574789 |
| ... | ...  | ...               | ...                   | ...          | ...    | ...       |
| 177 | 2016 | Jill Stein        | Green                 | 1457226      | loss   | 1.073699  |
| 178 | 2020 | Joseph Biden      | Democratic            | 81268924     | win    | 51.311515 |
| 179 | 2020 | Donald Trump      | Republican            | 74216154     | loss   | 46.858542 |
| 180 | 2020 | Jo Jorgensen      | Libertarian           | 1865724      | loss   | 1.177979  |
| 181 | 2020 | Howard Hawkins    | Green                 | 405035       | loss   | 0.255731  |

A **row** represents one record (i.e. an observation)

A **column** represents some characteristic, or feature, of that observation (here, the political party of that person).

A tabular **dataset** is **tidy** when each column corresponds to one **variable** in the **dataset**, each row corresponds to one **observation**, and all **variables** in the **dataset** have the same **unit of observation**.

# Intro To Pandas

---

Lecture 01, CSCI 3022

- Intros & Logistics
- What is Data Science?
- Exploratory Data Analysis & Wrangling
- Intro To Pandas
  - Jupyter Notebook Demo: EDA using Pandas

# Introducing the Standard Python Data Science Tool: Pandas

The Python Data  
Analysis Library



```
# data manipulation  
import pandas as pd
```

- **Pandas** (derived from Panel Data) is a Data Analysis library to make data cleaning and analysis fast and convenient in Python.
- Pandas adopts many coding idioms from NumPy, the biggest difference is that ***pandas is designed for working with tabular or heterogeneous data.***
  - NumPy by contrast is best suited for working with homogenous numerical data.

Tabular data is one of the most common data formats.

- Will be our primary focus in CSCI 3022

## Introducing the Standard Python Data Science Tool: pandas

---

Using `pandas`, we can:

- Arrange data in a tidy tabular format.
- Extract useful information filtered by specific conditions.
- Operate on data to gain new insights.
- Apply `NumPy` functions to our data
- Perform vectorized computations to speed up our analysis.

`pandas` is the standard tool across research and industry for working with tabular data.

The first week of this course will serve as a "bootcamp" in helping you build familiarity with operating on data with `pandas`.

# Pandas Data Structures

There are three fundamental data structures in pandas:

- **Series**: 1D labeled array data. I usually think of it as columnar data.
- **Data Frame**: 2D tabular data with both row and column labels
- **Index**: A sequence of row/column labels.

**Data Frame**

|   | Candidate | Party      | %    | Year | Result |
|---|-----------|------------|------|------|--------|
| 0 | Obama     | Democratic | 52.9 | 2008 | win    |
| 1 | McCain    | Republican | 45.7 | 2008 | loss   |
| 2 | Obama     | Democratic | 51.1 | 2012 | win    |
| 3 | Romney    | Republican | 47.2 | 2012 | loss   |
| 4 | Clinton   | Democratic | 48.2 | 2016 | loss   |
| 5 | Trump     | Republican | 46.1 | 2016 | win    |

**Series named "Candidate"**

|                                |         |
|--------------------------------|---------|
| 0                              | Obama   |
| 1                              | McCain  |
| 2                              | Obama   |
| 3                              | Romney  |
| 4                              | Clinton |
| 5                              | Trump   |
| Name: Candidate, dtype: object |         |

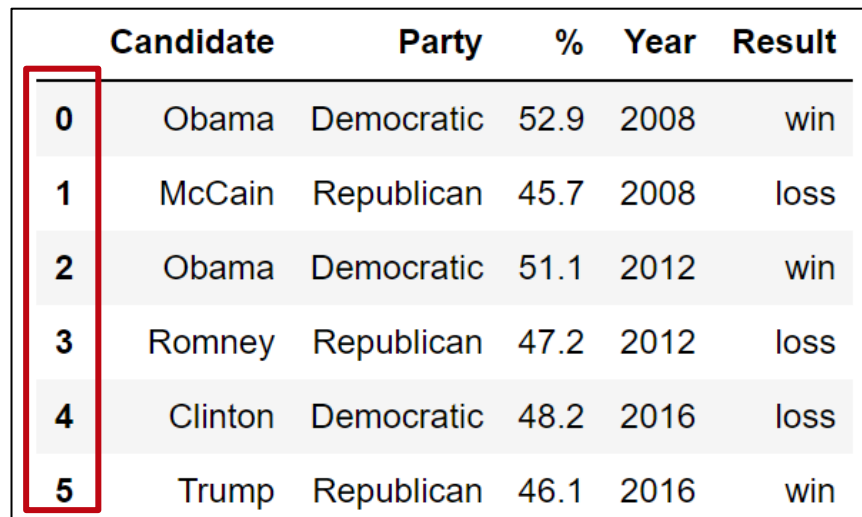
Index

# The Relationship Between DataFrames, Series, and Indices

We can think of a **DataFrame** as a collection of **Series** that all share the same **Index**.

- Candidate, Party, %, Year, and Result **Series** all share an **Index** from 0 to 5.

Candidate Series    Party Series    % Series    Year Series    Result Series



|   | Candidate | Party      | %    | Year | Result |
|---|-----------|------------|------|------|--------|
| 0 | Obama     | Democratic | 52.9 | 2008 | win    |
| 1 | McCain    | Republican | 45.7 | 2008 | loss   |
| 2 | Obama     | Democratic | 51.1 | 2012 | win    |
| 3 | Romney    | Republican | 47.2 | 2012 | loss   |
| 4 | Clinton   | Democratic | 48.2 | 2016 | loss   |
| 5 | Trump     | Republican | 46.1 | 2016 | win    |

## Lesson Learning Objectives:

- Identify 5 key data properties to consider when doing Exploratory Data Analysis and implement using demo data

- Intros & Logistics
- What is Data Science?
- Exploratory Data Analysis & Wrangling
- Intro to Pandas
  - Jupyter Notebook Demo: EDA using Pandas

# Jupyter Demo

---

Lecture 01, CSCI 3022



# Demo Slides

## CSV: Comma-Separated Values

Election Data in the US

CSV is a very common **tabular file format**.

- **Records** (rows) are delimited by a newline: `'\n'`, `"\r\n"`
- **Fields** (columns) are delimited by commas: `' , '`

Pandas: `pd.read_csv(header=...)`

| Records/Rows | Fields/Attributes/Features/Columns |              |               |
|--------------|------------------------------------|--------------|---------------|
|              | Year                               | Candidate    | ...           |
|              | 0                                  | 2024         | Kamala Harris |
| 1            | 2024                               | Donald Trump | ...           |

# Creating a DataFrame

Many approaches exist for creating a DataFrame.

## 1). From a CSV file.

```
elections = pd.read_csv("data/elections.csv")
```

|     | Year | Candidate         | Party                 | Popular vote | Result | %         |
|-----|------|-------------------|-----------------------|--------------|--------|-----------|
| 0   | 1824 | Andrew Jackson    | Democratic-Republican | 151271       | loss   | 57.210122 |
| 1   | 1824 | John Quincy Adams | Democratic-Republican | 113142       | win    | 42.789878 |
| 2   | 1828 | Andrew Jackson    | Democratic            | 642806       | win    | 56.203927 |
| 3   | 1828 | John Quincy Adams | National Republican   | 500897       | loss   | 43.796073 |
| 4   | 1832 | Andrew Jackson    | Democratic            | 702735       | win    | 54.574789 |
| ... | ...  | ...               | ...                   | ...          | ...    | ...       |
| 177 | 2016 | Jill Stein        | Green                 | 1457226      | loss   | 1.073699  |
| 178 | 2020 | Joseph Biden      | Democratic            | 81268924     | win    | 51.311515 |
| 179 | 2020 | Donald Trump      | Republican            | 74216154     | loss   | 46.858542 |
| 180 | 2020 | Jo Jorgensen      | Libertarian           | 1865724      | loss   | 1.177979  |
| 181 | 2020 | Howard Hawkins    | Green                 | 405035       | loss   | 0.255731  |

```
elections = pd.read_csv("data/elections.csv", index_col="Year")
```

|      | Candidate         | Party                 | Popular vote | Result | %         |
|------|-------------------|-----------------------|--------------|--------|-----------|
| Year |                   |                       |              |        |           |
| 1824 | Andrew Jackson    | Democratic-Republican | 151271       | loss   | 57.210122 |
| 1824 | John Quincy Adams | Democratic-Republican | 113142       | win    | 42.789878 |
| 1828 | Andrew Jackson    | Democratic            | 642806       | win    | 56.203927 |
| 1828 | John Quincy Adams | National Republican   | 500897       | loss   | 43.796073 |
| 1832 | Andrew Jackson    | Democratic            | 702735       | win    | 54.574789 |
| ...  | ...               | ...                   | ...          | ...    | ...       |
| 2016 | Jill Stein        | Green                 | 1457226      | loss   | 1.073699  |
| 2020 | Joseph Biden      | Democratic            | 81268924     | win    | 51.311515 |
| 2020 | Donald Trump      | Republican            | 74216154     | loss   | 46.858542 |
| 2020 | Jo Jorgensen      | Libertarian           | 1865724      | loss   | 1.177979  |
| 2020 | Howard Hawkins    | Green                 | 405035       | loss   | 0.255731  |

The DataFrame elections

2). See Supporting Materials for other methods to create DataFrame

The DataFrame elections with "Year" as Index



# Supporting Materials

---

Lecture 01, CSCI 3022

**Supporting Materials: Pandas Data Structures:**

**Series**

**DataFrames**

**Indices**

## Learning Objectives

- Understand the relationship between Series, DataFrames and Indices in Pandas
- Create, filter and perform operations on Series

# Pandas Data Structures: Series

---

- Introduction to Series

There are three fundamental data structures in pandas:

- **Series**: 1D labeled array data. I usually think of it as columnar data.
- **Data Frame**: 2D tabular data with both row and column labels
- **Index**: A sequence of row/column labels.

**Data Frame**

|   | Candidate | Party      | %    | Year | Result |
|---|-----------|------------|------|------|--------|
| 0 | Obama     | Democratic | 52.9 | 2008 | win    |
| 1 | McCain    | Republican | 45.7 | 2008 | loss   |
| 2 | Obama     | Democratic | 51.1 | 2012 | win    |
| 3 | Romney    | Republican | 47.2 | 2012 | loss   |
| 4 | Clinton   | Democratic | 48.2 | 2016 | loss   |
| 5 | Trump     | Republican | 46.1 | 2016 | win    |

**Series named “Candidate”**

|                                |         |
|--------------------------------|---------|
| 0                              | Obama   |
| 1                              | McCain  |
| 2                              | Obama   |
| 3                              | Romney  |
| 4                              | Clinton |
| 5                              | Trump   |
| Name: Candidate, dtype: object |         |

Index

# Series

A **Series** is a 1-dimensional array-like object. It contains:

- A sequence of **values** of the same type.
- A sequence of data labels, called the **index**.

**pd** is the conventional alias for **pandas**

```
import pandas as pd  
s = pd.Series(["welcome", "to", "cspb 3022"])
```

|   |           |
|---|-----------|
| 0 | welcome   |
| 1 | to        |
| 2 | CSPB 3022 |

dtype: object

**Index**, accessed by calling `s.index`

**Values**, accessed by calling `s.values`

## Series - Custom Index

- We can provide index labels for items in a Series by passing an index list.

```
s = pd.Series([-1, 10, 2], index = ["a", "b", "c"])
```

```
a    -1  
b    10  
c     2  
dtype: int64
```

```
s.index
```

```
Index(['a', 'b', 'c'], dtype='object')
```

- A Series index can also be changed.

```
s.index = ["first", "second", "third"]
```

```
first    -1  
second   10  
third     2  
dtype: int64
```

```
s.index
```

```
Index(['first', 'second', 'third'], dtype='object')
```

## Selection in Series

- We can select a single value or a set of values in a Series using:
  - A single label
  - A list of labels
  - A filtering condition

```
s = pd.Series([4, -2, 0, 6], index = ["a", "b", "c", "d"])
```

```
a      4
b     -2
c      0
d      6
dtype: int64
```



## Selection in Series

- We can select a single value or a set of values in a Series using:
  - **A single label**
  - A list of labels
  - A filtering condition

```
s = pd.Series([4, -2, 0, 6], index = ["a", "b", "c", "d"])
```

```
a    4
b   -2
c    0
d    6
dtype: int64
```

```
s["a"]
```

```
4
```

## Selection in Series

- We can select a single value or a set of values in a Series using:
  - A single label
  - **A list of labels**
  - A filtering condition

```
s = pd.Series([4, -2, 0, 6], index = ["a", "b", "c", "d"])
```

```
a      4
b     -2
c      0
d      6
dtype: int64
```

```
s[["a", "c"]]
```

```
a      4
c      0
dtype: int64
```

## Selection in Series

- We can select a single value or a set of values in a **Series** using:
  - A single label
  - A list of labels
  - **A filtering condition**

```
s = pd.Series([4, -2, 0, 6], index = ["a", "b", "c", "d"])
```

```
a    4
b   -2
c    0
d    6
dtype: int64
```

- Say we want to select values in the **Series** that satisfy a particular condition:
  - 1) Apply a boolean condition to the **Series**. This creates a **new Series of boolean values**.
  - 2) Index into our **Series** using this boolean condition. **pandas** will select only the entries in the **Series** that satisfy the condition.

```
s > 0
```

```
a    True
b   False
c   False
d    True
dtype: bool
```

```
s[s > 0]
```

```
a    4
d    6
dtype: int64
```

## Practice Question

---

What is the output of the following code?

```
example = pd.Series([4, 5, 6], index=["one", "two", "three"])  
example[example > 4].values
```

A). `array([4, 5, 6])`

C). `array([5, 6])`

`Index(['two', 'three'], dtype='object')`

D). `[5, 6]`

B).

E). None of these

## Practice Question

What is the output of the following code?

```
example = pd.Series([4, 5, 6], index=["one", "two", "three"])  
example[example > 4].values
```

A). `array([4, 5, 6])`

`Index(['two', 'three'], dtype='object')`

B).

C). `array([5, 6])`

D). `[5, 6]`

E). None of these

# DataFrames of Series!

Typically, we will work with **Series** using the perspective that they are columns in a **DataFrame**.

We can think of a **DataFrame** as a collection of **Series** that all share the same **Index**.

0

1824

1

1824

2

1828

3

1828

4

1832

...

177

2016

178

2020

179

2020

180

2020

181

2020

Name: Year,

0

Andrew Jackson

1

John Quincy Adams

2

Andrew Jackson

3

John Quincy Adams

4

Andrew Jackson

...

177

Jill Stein

178

Joseph Biden

179

Donald Trump

180

Jo Jorgensen

181

Howard Hawkins

Name: Candidate,

+

[...]

➔

|     | Year | Candidate         | Party                 | Popular vote | Result | %         |
|-----|------|-------------------|-----------------------|--------------|--------|-----------|
| 0   | 1824 | Andrew Jackson    | Democratic-Republican | 151271       | loss   | 57.210122 |
| 1   | 1824 | John Quincy Adams | Democratic-Republican | 113142       | win    | 42.789878 |
| 2   | 1828 | Andrew Jackson    | Democratic            | 642806       | win    | 56.203927 |
| 3   | 1828 | John Quincy Adams | National Republican   | 500897       | loss   | 43.796073 |
| 4   | 1832 | Andrew Jackson    | Democratic            | 702735       | win    | 54.574789 |
| ... | ...  | ...               | ...                   | ...          | ...    | ...       |
| 177 | 2016 | Jill Stein        | Green                 | 1457226      | loss   | 1.073699  |
| 178 | 2020 | Joseph Biden      | Democratic            | 81268924     | win    | 51.311515 |
| 179 | 2020 | Donald Trump      | Republican            | 74216154     | loss   | 46.858542 |
| 180 | 2020 | Jo Jorgensen      | Libertarian           | 1865724      | loss   | 1.177979  |
| 181 | 2020 | Howard Hawkins    | Green                 | 405035       | loss   | 0.255731  |

The Series "Year"

The Series "Candidate"

The DataFrame elections



## Learning Objectives

- Understand the relationship between DataFrames and Indices in Pandas
- Create DataFrames
- Manipulate indices

# Pandas Data Structures

---

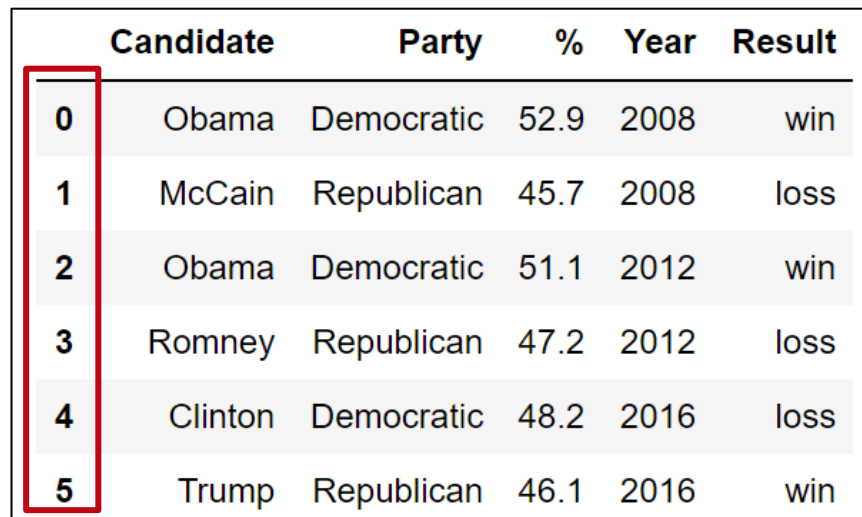
- Introduction to DataFrames
- Indices

# The Relationship Between DataFrames, Series, and Indices

We can think of a **DataFrame** as a collection of **Series** that all share the same **Index**.

- Candidate, Party, %, Year, and Result **Series** all share an **Index** from 0 to 5.

Candidate Series    Party Series    % Series    Year Series    Result Series



|   | Candidate | Party      | %    | Year | Result |
|---|-----------|------------|------|------|--------|
| 0 | Obama     | Democratic | 52.9 | 2008 | win    |
| 1 | McCain    | Republican | 45.7 | 2008 | loss   |
| 2 | Obama     | Democratic | 51.1 | 2012 | win    |
| 3 | Romney    | Republican | 47.2 | 2012 | loss   |
| 4 | Clinton   | Democratic | 48.2 | 2016 | loss   |
| 5 | Trump     | Republican | 46.1 | 2016 | win    |



# The DataFrame API

---

The API for the **DataFrame** class is enormous.

- API: "Application Programming Interface".
- The API is the set of abstractions supported by the class.

Full documentation is at

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html>

- We will only consider a tiny portion of this API.

We want you to get familiar with the real world programming practice of... Googling!

- Answers to your questions are often found in the **pandas** documentation, Stack Overflow, etc.

## Creating a DataFrame

---

The syntax of creating **DataFrame** is:

```
pandas.DataFrame(data, index, columns)
```

Many approaches exist for creating a **DataFrame**. Here, we will go over the most popular ones.

- From a CSV file.
- Using a list and column name(s).
- From a dictionary.
- From a **Series**.

# Creating a DataFrame

The syntax of creating `DataFrame` is:

```
pandas.DataFrame(data, index, columns)
```

Many approaches exist for creating a `DataFrame`. Here, we will go over the most popular ones.

- **From a CSV file.**
- Using a list and column name(s).
- From a dictionary.
- From a **Series**.

```
elections = pd.read_csv("data/elections.csv")
```

|     | Year | Candidate         | Party                 | Popular vote | Result | %         |
|-----|------|-------------------|-----------------------|--------------|--------|-----------|
| 0   | 1824 | Andrew Jackson    | Democratic-Republican | 151271       | loss   | 57.210122 |
| 1   | 1824 | John Quincy Adams | Democratic-Republican | 113142       | win    | 42.789878 |
| 2   | 1828 | Andrew Jackson    | Democratic            | 642806       | win    | 56.203927 |
| 3   | 1828 | John Quincy Adams | National Republican   | 500897       | loss   | 43.796073 |
| 4   | 1832 | Andrew Jackson    | Democratic            | 702735       | win    | 54.574789 |
| ... | ...  | ...               | ...                   | ...          | ...    | ...       |
| 177 | 2016 | Jill Stein        | Green                 | 1457226      | loss   | 1.073699  |
| 178 | 2020 | Joseph Biden      | Democratic            | 81268924     | win    | 51.311515 |
| 179 | 2020 | Donald Trump      | Republican            | 74216154     | loss   | 46.858542 |
| 180 | 2020 | Jo Jorgensen      | Libertarian           | 1865724      | loss   | 1.177979  |
| 181 | 2020 | Howard Hawkins    | Green                 | 405035       | loss   | 0.255731  |

The `DataFrame` `elections`

# Creating a DataFrame

The syntax of creating DataFrame is:

```
pandas.DataFrame(data, index, columns)
```

Many approaches exist for creating a DataFrame. Here, we will go over the most popular ones.

- **From a CSV file.** elections = pd.read\_csv("data/elections.csv", index\_col="Year")
- Using a list and column name(s).
- From a dictionary.
- From a Series.

|      | Candidate         | Party                 | Popular vote | Result | %         |
|------|-------------------|-----------------------|--------------|--------|-----------|
| Year |                   |                       |              |        |           |
| 1824 | Andrew Jackson    | Democratic-Republican | 151271       | loss   | 57.210122 |
| 1824 | John Quincy Adams | Democratic-Republican | 113142       | win    | 42.789878 |
| 1828 | Andrew Jackson    | Democratic            | 642806       | win    | 56.203927 |
| 1828 | John Quincy Adams | National Republican   | 500897       | loss   | 43.796073 |
| 1832 | Andrew Jackson    | Democratic            | 702735       | win    | 54.574789 |
| ...  | ...               | ...                   | ...          | ...    | ...       |
| 2016 | Jill Stein        | Green                 | 1457226      | loss   | 1.073699  |
| 2020 | Joseph Biden      | Democratic            | 81268924     | win    | 51.311515 |
| 2020 | Donald Trump      | Republican            | 74216154     | loss   | 46.858542 |
| 2020 | Jo Jorgensen      | Libertarian           | 1865724      | loss   | 1.177979  |
| 2020 | Howard Hawkins    | Green                 | 405035       | loss   | 0.255731  |

The DataFrame elections with "Year" as Index



## Creating a DataFrame

Many approaches exist for creating a **DataFrame**. Here, we will go over the most popular ones.

- From a CSV file.
- **Using a list and column name(s).**
- From a dictionary.
- From a **Series**.

```
pd.DataFrame([1, 2, 3],  
             columns=["Numbers"])
```

| Numbers |   |
|---------|---|
| 0       | 1 |
| 1       | 2 |
| 2       | 3 |

```
pd.DataFrame([[1, "one"], [2, "two"]],  
             columns = ["Number", "Description"])
```

| Number Description |   |     |
|--------------------|---|-----|
| 0                  | 1 | one |
| 1                  | 2 | two |

# Creating a DataFrame

Many approaches exist for creating a **DataFrame**. Here, we will go over the most popular ones.

- From a CSV file.
- Using a list and column name(s).
- **From a dictionary.**
- From a **Series**.

Wait, what's a dictionary?

**Dictionary in Python**  PYnative.com

Unordered collections of unique values stored in (Key-Value) pairs.

```
d = {'a': 10, 'b': 20, 'c': 30}
```

↑            ↑            ↑  
d['a']    d['b']    d['c']

- ✓ **Unordered:** The items in dict are stored without any index value
- ✓ **Unique:** Keys in dictionaries should be Unique
- ✓ **Mutable:** We can add/Modify/Remove key-value after the creation

```
d2={"Fruit":["Strawberry",  
"Orange"], "Price": [5.49, 3.99]}
```

## Creating a DataFrame

Many approaches exist for creating a **DataFrame**. Here, we will go over the most popular ones.

- From a CSV file.
- Using a list and column name(s).
- **From a dictionary.**
- From a **Series**.

```
pd.DataFrame({"Fruit":["Strawberry", "Orange"],  
             "Price": [5.49, 3.99]})
```

Specify columns of the **DataFrame**

```
pd.DataFrame([{"Fruit":"Strawberry", "Price":5.49},  
             {"Fruit":"Orange", "Price":3.99}])
```

|   | Fruit      | Price |
|---|------------|-------|
| 0 | Strawberry | 5.49  |
| 1 | Orange     | 3.99  |

Specify rows of the **DataFrame**

## Creating a DataFrame

Many approaches exist for creating a **DataFrame**. Here, we will go over the most popular ones.

- From a CSV file.
- Using a list and column name(s).
- From a dictionary.
- **From a Series.**

```
s_a = pd.Series(["a1", "a2", "a3"], index = ["r1", "r2", "r3"])  
s_b = pd.Series(["b1", "b2", "b3"], index = ["r1", "r2", "r3"])
```

```
pd.DataFrame({"A-column":s_a, "B-column":s_b})
```

|    | A-column | B-column |
|----|----------|----------|
| r1 | a1       | b1       |
| r2 | a2       | b2       |
| r3 | a3       | b3       |

```
pd.DataFrame(s_a)
```

```
s_a.to_frame()
```

|    | 0  |
|----|----|
| r1 | a1 |
| r2 | a2 |
| r3 | a3 |



# Indices Are Not Necessarily Row Numbers

A **Row Index** (a.k.a. row labels) can also:

- Be non-numeric.
- Have a name, e.g. "Candidate".

```
# Creating a DataFrame from a CSV file and specifying the Index column
elections = pd.read_csv("data/elections.csv", index_col = "Candidate")
```

| Candidate         | Year | Party                 | Popular vote | Result | %         |
|-------------------|------|-----------------------|--------------|--------|-----------|
| Andrew Jackson    | 1824 | Democratic-Republican | 151271       | loss   | 57.210122 |
| John Quincy Adams | 1824 | Democratic-Republican | 113142       | win    | 42.789878 |
| Andrew Jackson    | 1828 | Democratic            | 642806       | win    | 56.203927 |
| John Quincy Adams | 1828 | National Republican   | 500897       | loss   | 43.796073 |
| Andrew Jackson    | 1832 | Democratic            | 702735       | win    | 54.574789 |

# Indices Are Not Necessarily Unique

The row labels that constitute an index do not have to be unique.

- Left: The **index** values are all unique and numeric, acting as a row number.
- Right: The **index** values are named and non-unique.

|   | Candidate | Party      | %    | Year | Result |
|---|-----------|------------|------|------|--------|
| 0 | Obama     | Democratic | 52.9 | 2008 | win    |
| 1 | McCain    | Republican | 45.7 | 2008 | loss   |
| 2 | Obama     | Democratic | 51.1 | 2012 | win    |
| 3 | Romney    | Republican | 47.2 | 2012 | loss   |
| 4 | Clinton   | Democratic | 48.2 | 2016 | loss   |
| 5 | Trump     | Republican | 46.1 | 2016 | win    |

|      | Candidate | Party      | %    | Result |
|------|-----------|------------|------|--------|
| Year |           |            |      |        |
| 2008 | Obama     | Democratic | 52.9 | win    |
| 2008 | McCain    | Republican | 45.7 | loss   |
| 2012 | Obama     | Democratic | 51.1 | win    |
| 2012 | Romney    | Republican | 47.2 | loss   |
| 2016 | Clinton   | Democratic | 48.2 | loss   |
| 2016 | Trump     | Republican | 46.1 | win    |

# Modifying Indices

- We can select a new column and set it as the index of the **DataFrame**.

Example: Setting the index to the "Party" column.

```
elections = elections.set_index("Party")
```

|                       | Candidate         | Year | Popular vote | Result | %         |
|-----------------------|-------------------|------|--------------|--------|-----------|
| Party                 |                   |      |              |        |           |
| Democratic-Republican | Andrew Jackson    | 1824 | 151271       | loss   | 57.210122 |
| Democratic-Republican | John Quincy Adams | 1824 | 113142       | win    | 42.789878 |
| Democratic            | Andrew Jackson    | 1828 | 642806       | win    | 56.203927 |
| National Republican   | John Quincy Adams | 1828 | 500897       | loss   | 43.796073 |
| Democratic            | Andrew Jackson    | 1832 | 702735       | win    | 54.574789 |
| ...                   | ...               | ...  | ...          | ...    | ...       |
| Green                 | Jill Stein        | 2016 | 1457226      | loss   | 1.073699  |
| Democratic            | Joseph Biden      | 2020 | 81268924     | win    | 51.311515 |
| Republican            | Donald Trump      | 2020 | 74216154     | loss   | 46.858542 |
| Libertarian           | Jo Jorgensen      | 2020 | 1865724      | loss   | 1.177979  |
| Green                 | Howard Hawkins    | 2020 | 405035       | loss   | 0.255731  |

# Resetting the Index

- We can change our mind and reset the **Index** back to the default list of integers.

```
elections = elections.reset_index()
```

|                       | Candidate         | Year | Popular vote | Result | %         |
|-----------------------|-------------------|------|--------------|--------|-----------|
| Party                 |                   |      |              |        |           |
| Democratic-Republican | Andrew Jackson    | 1824 | 151271       | loss   | 57.210122 |
| Democratic-Republican | John Quincy Adams | 1824 | 113142       | win    | 42.789878 |
| Democratic            | Andrew Jackson    | 1828 | 642806       | win    | 56.203927 |
| National Republican   | John Quincy Adams | 1828 | 500897       | loss   | 43.796073 |
| Democratic            | Andrew Jackson    | 1832 | 702735       | win    | 54.574789 |
| ...                   | ...               | ...  | ...          | ...    | ...       |
| Green                 | Jill Stein        | 2016 | 1457226      | loss   | 1.073699  |
| Democratic            | Joseph Biden      | 2020 | 81268924     | win    | 51.311515 |
| Republican            | Donald Trump      | 2020 | 74216154     | loss   | 46.858542 |
| Libertarian           | Jo Jorgensen      | 2020 | 1865724      | loss   | 1.177979  |
| Green                 | Howard Hawkins    | 2020 | 405035       | loss   | 0.255731  |

|     | Candidate         | Year | Party                 | Popular vote | Result | %         |
|-----|-------------------|------|-----------------------|--------------|--------|-----------|
| 0   | Andrew Jackson    | 1824 | Democratic-Republican | 151271       | loss   | 57.210122 |
| 1   | John Quincy Adams | 1824 | Democratic-Republican | 113142       | win    | 42.789878 |
| 2   | Andrew Jackson    | 1828 | Democratic            | 642806       | win    | 56.203927 |
| 3   | John Quincy Adams | 1828 | National Republican   | 500897       | loss   | 43.796073 |
| 4   | Andrew Jackson    | 1832 | Democratic            | 702735       | win    | 54.574789 |
| ... | ...               | ...  | ...                   | ...          | ...    | ...       |
| 177 | Jill Stein        | 2016 | Green                 | 1457226      | loss   | 1.073699  |
| 178 | Joseph Biden      | 2020 | Democratic            | 81268924     | win    | 51.311515 |
| 179 | Donald Trump      | 2020 | Republican            | 74216154     | loss   | 46.858542 |
| 180 | Jo Jorgensen      | 2020 | Libertarian           | 1865724      | loss   | 1.177979  |
| 181 | Howard Hawkins    | 2020 | Green                 | 405035       | loss   | 0.255731  |



# Column Names Are Usually Unique!

Column names in **pandas** are almost always unique.

- Example: Really shouldn't have two columns named "Candidate".

|   | Candidate | Party      | %    | Year | Result |
|---|-----------|------------|------|------|--------|
| 0 | Obama     | Democratic | 52.9 | 2008 | win    |
| 1 | McCain    | Republican | 45.7 | 2008 | loss   |
| 2 | Obama     | Democratic | 51.1 | 2012 | win    |
| 3 | Romney    | Republican | 47.2 | 2012 | loss   |
| 4 | Clinton   | Democratic | 48.2 | 2016 | loss   |
| 5 | Trump     | Republican | 46.1 | 2016 | win    |

# Accessing a DataFrame's columns and row indices: .index and .columns

```
elections = pd.read_csv("data/elections.csv", index_col="Year")
```

|      | Candidate         | Party                 | Popular vote | Result | %         |
|------|-------------------|-----------------------|--------------|--------|-----------|
| Year |                   |                       |              |        |           |
| 1824 | Andrew Jackson    | Democratic-Republican | 151271       | loss   | 57.210122 |
| 1824 | John Quincy Adams | Democratic-Republican | 113142       | win    | 42.789878 |
| 1828 | Andrew Jackson    | Democratic            | 642806       | win    | 56.203927 |
| 1828 | John Quincy Adams | National Republican   | 500897       | loss   | 43.796073 |
| 1832 | Andrew Jackson    | Democratic            | 702735       | win    | 54.574789 |
| ...  | ...               | ...                   | ...          | ...    | ...       |
| 2016 | Jill Stein        | Green                 | 1457226      | loss   | 1.073699  |
| 2020 | Joseph Biden      | Democratic            | 81268924     | win    | 51.311515 |
| 2020 | Donald Trump      | Republican            | 74216154     | loss   | 46.858542 |
| 2020 | Jo Jorgensen      | Libertarian           | 1865724      | loss   | 1.177979  |
| 2020 | Howard Hawkins    | Green                 | 405035       | loss   | 0.255731  |

```
elections.index
Index([1824, 1824, 1828, 1828, 1832, 1832, 1832, 1836, 1836,
      ...
      2016, 2016, 2016, 2016, 2016, 2016, 2020, 2020, 2020, 2020],
      dtype='int64', name='Year', length=182)
```

```
elections.columns
Index(['Candidate', 'Party', 'Popular vote', 'Result', '%'], dtype='object')
```

The DataFrame `elections` with `"Year"` as Index